

Welche der 6 R-Strategien ist für die verschiedenen Komponenten von frag.jetzt (Frontend-PWA, Spring Boot Backend, PostgreSQL-Datenbank, AI-Services) jeweils am besten geeignet?

Cloud Migrationsstrategien im Vergleich

Yannick Diehl

14. Januar 2026

Zusammenfassung

Die Modernisierung bestehender Softwaresysteme erfordert Entscheidungen unter technischen, organisatorischen und betrieblichen Randbedingungen. Anhand der Webanwendung **frag.jetzt** untersucht diese Arbeit, welche der „6 R“-Strategien (u. a. Rehost, Replatform, Refactor/Re-architect, Rebuild, Replace) für die jeweiligen Systemkomponenten am geeignetsten ist: Frontend als Progressive Web App, Spring-Boot-Backend, PostgreSQL-Datenbank und angebundene AI-Services. Methodisch wird eine Fallstudie mit einem expliziten Bewertungsrahmen kombiniert, der Strategien je Komponente entlang definierter Kriterien (z. B. Wartbarkeit, Sicherheitsanforderungen, Integrationsaufwand, Betriebsrisiko, Kosten und Lock-in) vergleicht. Ergebnis ist eine nachvollziehbar begründete Strategiezurordnung pro Komponente einschließlich identifizierter Risiken und Implikationen für eine schrittweise Umsetzung.

ist eine systematische Entscheidung *pro Komponente*: Je nach technischem Zustand, fachlichem Nutzen, Risiko und Kopplungsgrad wird eine andere Vorgehensweise empfohlen, etwa *Rehost* (Lift-and-Shift), *Replatform* (geringe technische Anpassungen), *Refactor* bzw. *Re-architect* (tiefgreifende Anpassungen), *Repurchase/Replace* (Wechsel zu einem anderen Produkt bzw. SaaS), *Retire* (Außerbetriebnahme) oder *Retain* (Beibehaltung) [1].

Diese Bachelorarbeit untersucht diese Fragestellung anhand der Anwendung **frag.jetzt**. Das System lässt sich in klar unterscheidbare Komponenten zerlegen: ein Frontend als Progressive Web App (PWA), ein Spring-Boot-Backend als Anwendungslogik und API-Schicht, eine PostgreSQL-Datenbank als persistente Datenhaltung sowie angebundene AI-Services, die zusätzliche Anforderungen an Datenschutz, Kostenkontrolle und Betriebsstabilität mitbringen (u. a. durch die Abhängigkeit von externen Modellen und deren Update-Zyklen).

1 Einleitung

Spätestens seit dem starken Anstieg regulatorischer Anforderungen, Sicherheitsvorfälle und der dynamischen Entwicklung datengetriebener Funktionen (insbesondere im Umfeld von AI-Services) steht in vielen Organisationen nicht mehr *ob*, sondern *wie* bestehende Software modernisiert werden soll. Gleichzeitig ist die Ausgangslage häufig durch knappe Budgets, begrenzte Personalressourcen und einen laufenden Betrieb geprägt, der nicht beliebig unterbrochen werden kann. Modernisierung muss daher als kontrollierter, risikobewusster Veränderungsprozess verstanden werden: Ziel ist nicht ein maximal modernes Zielbild um jeden Preis, sondern eine begründete Veränderung mit klarer Priorisierung.

Für diese Entscheidungsfindung haben sich in der Praxis Strategierahmen etabliert, die Modernisierung als Menge diskreter Optionen modellieren. Ein verbreitetes Modell sind die sogenannten „6 R“-Strategien, die unter anderem in Cloud-Migrations- und Portfolio-Assessment-Ansätzen beschrieben werden. Kernidee

1.1 Forschungsfrage und Zielsetzung

Die Arbeit beantwortet die Forschungsfrage:

*Welche der 6 R-Strategien ist für die verschiedenen Komponenten von **frag.jetzt** jeweils am besten geeignet?*

Ziel ist eine nachvollziehbar begründete Zuordnung einer Strategie *pro Komponente*. Die Begründung erfolgt anhand eines Kriterienkatalogs, der sowohl technische als auch organisatorische Aspekte berücksichtigt, insbesondere Wartbarkeit, Sicherheitsanforderungen, Integrationsaufwand, Performance, Betriebskosten und erwartete Weiterentwicklung. Die Ergebnisse sollen so formuliert sein, dass sie als Entscheidungsgrundlage für eine schrittweise Modernisierung nutzbar sind.

1.2 Aufbau der Arbeit

Die Arbeit gliedert sich in vier Teile: (1) theoretische Fundierung der Modernisierungsstrategien und relevanter Qualitätsattribute, (2) Beschreibung des Systemkontexts und der betrachteten Komponenten,

(3) methodisches Vorgehen zur Bewertung und Auswahl der Strategien sowie (4) Ergebnisdarstellung als begründete Zuordnung je Komponente inklusive Umsetzungsfahrplan und Reflexion der Grenzen.

2 Theoretische Fundierung

2.1 Software-Modernisierung als architektonische Entscheidungsaufgabe

Software-Modernisierung umfasst Maßnahmen, die technische und strukturelle Eigenschaften eines Systems verbessern, um langfristige Wartbarkeit, Anpassbarkeit und Betriebsfähigkeit zu sichern. Aus architektonischer Sicht ist Modernisierung eine Abfolge von Entscheidungen unter Unsicherheit: Jede Maßnahme verändert Kopplungen, Schnittstellen und Qualitätsmerkmale und kann dadurch neue Risiken oder Abhängigkeiten erzeugen. Architektur wird in diesem Sinne als *Trade-off* zwischen konkurrierenden Qualitätszielen verstanden, etwa zwischen schneller Lieferung und hoher Robustheit oder zwischen Kostenminimierung und Reduktion von Vendor-Lock-in [2, 3].

Ein zentraler Grundsatz ist, dass nicht alle Komponenten denselben Modernisierungsgrad benötigen. Komponenten unterscheiden sich bezüglich Änderungsrate, Sicherheitskritikalität, Skalierungsbedarf und Technologierisiko. Eine PWA ist beispielsweise stark durch Browser-APIs, Endgerätevielfalt und UX-Anforderungen geprägt, während eine Datenbank primär durch Datenintegrität, Migrationsrisiken und Betriebsstabilität dominiert wird. Folglich ist eine komponentenspezifische Strategieauswahl plausibler als eine monolithische „One-size-fits-all“-Modernisierung.

2.2 Die 6 R-Strategien

Das 6 R-Modell dient der strukturierten Portfolio-Entscheidung. AWS beschreibt einen eng verwandten Katalog von Migrationsstrategien (häufig als „7 Rs“ bezeichnet); in dieser Arbeit wird für die weitere Argumentation eine gebräuchliche 6 R-Variante verwendet, in der die zusätzliche Kategorie „Relocate/Verschieben“ nicht separat betrachtet wird [1]:

- **Rehost:** Migration mit minimalen Änderungen (z. B. Infrastrukturwechsel), primär zur schnellen Entlastung oder Standardisierung.
- **Replatform:** Migration mit begrenzten technischen Anpassungen, typischerweise zur Nutzung eines managed Dienstes oder zur Reduktion operativer Last.
- **Refactor / Re-architect:** Strukturelle Änderung der Anwendung (z. B. Modularisierung, Entkopplung, Cloud-Native-Patterns) zur Verbesserung von Skalierbarkeit, Wartbarkeit oder Resilienz.

- **Repurchase / Replace:** Wechsel zu einem anderen Produkt (z. B. SaaS) statt Weiterbetrieb der bestehenden Lösung.
- **Retire:** Außerbetriebnahme, wenn kein geschäftlicher Nutzen (mehr) besteht.
- **Retain:** Beibehaltung (vorerst keine Migration), z. B. bei Abhängigkeiten oder fehlender Priorität.

In der Praxis werden einzelne Begriffe unterschiedlich abgegrenzt (insbesondere *Refactor* vs. *Re-architect*). In dieser Arbeit werden beide als Spektrum tiefgreifender, architektureller Veränderungen verstanden, die über rein kosmetische Code-Verbesserungen hinausgehen.

2.3 Komponententypen und Qualitätsanforderungen

Für die Bewertung der Strategien werden Qualitätsanforderungen herangezogen, die in der Softwarearchitektur als entscheidungsleitend gelten [2, 3]. Für **frag.jetzt** sind insbesondere relevant:

- **Wartbarkeit und Änderbarkeit:** Aufwand zur Implementierung neuer Anforderungen ohne regressionsbedingte Risiken.
- **Sicherheit und Datenschutz:** Schutz vor Missbrauch, Einhaltung organisatorischer und rechtlicher Vorgaben.
- **Betriebsstabilität (Resilienz):** Fähigkeit, Fehler zu tolerieren, degradierte Zustände zu beherrschen und Wiederherstellung zu ermöglichen.
- **Kosten und Effizienz:** direkte Betriebskosten sowie indirekte Kosten durch Komplexität und organisatorischen Overhead.
- **Portabilität und Lock-in-Risiko:** Abhängigkeit von spezifischen Plattformen oder Diensten.

Im Kontext von AI-Services treten zusätzliche Risiken auf, etwa durch Modell-Updates, unvorhersehbare Antwortqualität, Datenschutzerfordernisse und die Notwendigkeit systematischer Risikosteuerung. Rahmenwerke wie der NIST AI RMF bieten hierfür Kategorien zur Risikoidentifikation und -behandlung, die sich als ergänzende Kriterien operationalisieren lassen [4].

2.4 Systemkontext: frag.jetzt als Fallstudie

frag.jetzt wird in dieser Arbeit als exemplarisches System verstanden, an dem sich die Strategieauswahl für typische Webanwendungs-Komponenten demonstrieren lässt. Die technische Aufteilung in Frontend (PWA), Backend (Spring Boot), Datenhaltung (PostgreSQL) und externe AI-Services ist in modernen Anwendungen häufig anzutreffen und ermöglicht eine getrennte Betrachtung. Für die Komponentenbeschreibung werden primär Architekturartefakte (z. B. Schnittstellen, Deployments, Datenflüsse) und betrieblich relevante Eigenschaften (z. B. Skalierungsprofile, Ausfallfolgen) herangezogen.

3 Methodik

3.1 Forschungsdesign

Methodisch wird eine qualitative Fallstudie mit einem formalisierten Bewertungsrahmen kombiniert. Die Fallstudie erlaubt es, kontextabhängige Faktoren (z. B. organisatorische Constraints, Betriebsrealität, technische Historie) zu berücksichtigen. Der Bewertungsrahmen sorgt dafür, dass die resultierende Strategieentscheidung nicht rein intuitiv erfolgt, sondern entlang expliziter Kriterien transparent begründet wird.

3.2 Bewertungsrahmen: Kriterienkatalog und Entscheidungsmatrix

Für jede Komponente wird eine Entscheidungsmatrix erstellt, in der die 6 R-Strategien entlang eines Kriterienkatalogs bewertet werden. Kriterien werden in mess- oder zumindest begründbar beobachtbare Indikatoren überführt (z. B. „Deployment-Frequenz“ als Proxy für Änderungsrate, „Anzahl externer Abhängigkeiten“ als Proxy für Integrationskomplexität). Die Bewertung erfolgt in einer ordinalen Skala (z. B. 1–5), ergänzt durch qualitative Begründungen.

Ein Beispiel für die Struktur einer solchen Matrix zeigt Tabelle 1.

Kriterium	Rehost	Replatform	Refactor	Replace
Betriebsrisiko	mittel	mittel	hoch	niedrig-mittel
Kosten (kurzfristig)	niedrig	niedrig-mittel	hoch	mittel
Wartbarkeit (langfristig)	niedrig	mittel	hoch	mittel-hoch
Lock-in-Risiko	niedrig	mittel	mittel	hoch

Tabelle 1: Schematische Entscheidungsmatrix (Beispielstruktur; Werte komponentenspezifisch)

Die Gewichtung der Kriterien wird explizit festgelegt (z. B. durch Experteneinschätzung oder vereinfachte Paarvergleiche). Dadurch kann transparent gezeigt werden, warum eine Strategie trotz hoher Implementierungskosten dennoch vorzugswürdig sein kann (etwa bei hohen Sicherheitsanforderungen).

3.3 Datenerhebung

Zur Datenerhebung werden folgende Quellen vorgese-
hen:

- **Architektur- und Codeanalyse:** Sichtung von Modulgrenzen, Schnittstellen, Datenmodellen, Abhängigkeiten und Build-/Deployment-Prozessen.
- **Betriebsdaten (sofern verfügbar):** Logs, Monitoring, Ausfallberichte, Latenz-/Lastprofile und Kostenindikatoren.
- **Dokumente und Standards:** technische Dokumentation zu PWA [5], Spring Boot [6] und PostgreSQL [7] sowie Risiko- und Governance-Rahmen für AI-Services [4].

- **Stakeholder-Interviews (optional):** Erhebung nicht-technischer Anforderungen (z. B. Releasezyklen, Compliance, Teamkompetenzen).

3.4 Auswertung und Validität

Die Auswertung erfolgt komponentenweise: Für jede Komponente werden (1) Ist-Zustand und Qualitätsanforderungen beschrieben, (2) Strategien anhand der Matrix bewertet und (3) eine Empfehlung mit Begründung und Risiken formuliert. Zur Erhöhung der Validität wird eine *Plausibilitätsprüfung* eingeplant: Die Empfehlungen werden gegen bekannte Architekturprinzipien sowie gegen typische Fallstricke moderner Architekturen gespiegelt (z. B. Überfragmentierung, operativer Overhead, unbeabsichtigte Lock-in-Effekte) [2, 3].

Grenzen der Methodik liegen vor allem in der Kontextabhängigkeit: Ergebnisse sind nicht ohne Weiteres auf andere Systeme übertragbar. Genau darin liegt jedoch zugleich der Beitrag einer Fallstudie, nämlich die nachvollziehbare Argumentation im konkreten Kontext.

4 Geplante Gliederung der Arbeit

Die folgende Gliederung beschreibt den vorgesehenen Aufbau der Bachelorarbeit und dient als Arbeitsplan. Kapitelnummern und Umfang sind als Zielwerte zu verstehen.

1. Einleitung

- Problemstellung und Motivation
- Forschungsfrage, Zielsetzung, Beitrag
- Vorgehensüberblick

2. Theoretische Grundlagen

- Software-Modernisierung und Architekturentscheidungen [2, 3]
- 6 R-Strategien als Portfolio-Ansatz [1]
- Qualitätsattribute und Bewertungskriterien

3. Systemkontext und Komponentenbeschreibung (frag.jetzt)

- Frontend-PWA: Architektur, Constraints und Release-/UX-Anforderungen [5]
- Spring-Boot-Backend: API-Schicht, Domänenlogik, Integrationen [6]
- PostgreSQL: Datenmodell, Migrations- und Betriebsaspekte [7]
- AI-Services: Abhängigkeiten, Datenschutz, Kosten und Risikoaspekte [4]

4. Methodik

- Kriterienkatalog, Gewichtung, Entscheidungsmatrix
- Datenerhebung und Auswertungsverfahren
- Validität, Limitationen

5. Ergebnisse: Strategieempfehlung je Komponente

- Begründete Zuordnung einer 6 R-Strategie pro Komponente

- Risikoanalyse und Annahmen
6. **Diskussion und Fazit**
- Implikationen für eine schrittweise Modernisierung
 - Grenzen der Ergebnisse und Ausblick

Glossar

6 R-Strategien:

Sammlung von Modernisierungsoptionen zur systematischen Entscheidung pro Anwendungskomponente (z. B. Rehost, Replatform, Refactor/Re-architect, Repurchase/Replace, Retire, Retain) [1].

AI-Service:

Externer oder interner Dienst, der KI-Funktionalität bereitstellt (z. B. LLM-API, Embedding-Service). Solche Dienste erfordern besonderes Risikomanagement, u. a. hinsichtlich Datenschutz, Nachvollziehbarkeit und Kosten [4].

Backend (Spring Boot):

Serverseitige Anwendungskomponente, die Geschäftslogik und Schnittstellen (typisch REST/HTTP) bereitstellt. In dieser Arbeit repräsentiert das Backend die API- und Domänenschicht [6].

Lock-in:

Bindung an spezifische Plattformen, APIs oder Dienste, die einen späteren Wechsel erschweren (technisch oder wirtschaftlich).

PWA (Progressive Web App):

Webanwendung, die nativeähnliche Eigenschaften (Offline-Fähigkeit, Installierbarkeit, Push) durch moderne Browser-Technologien erreicht [5].

PostgreSQL:

Relationales Datenbanksystem; im Kontext dieser Arbeit steht es für die persistente Datenerhaltung inklusive Transaktions- und Migrationsaspekten [7].

Refactor/Re-architect:

Tiefgreifende Umgestaltung einer Komponente zur Verbesserung zentraler Qualitätsattribute, z. B. Modularisierung, Entkopplung oder Cloud-Native-Patterns [1, 3].

Rehost:

Migration mit minimalen Änderungen (Lift-and-Shift), primär zur schnellen Verlagerung und Standardisierung [1].

Replatform:

Migration mit begrenzten technischen Anpassungen, häufig um Betriebsaufwand zu reduzieren oder managed Dienste zu nutzen [1].

Repurchase/Replace:

Wechsel zu einem anderen Produkt (z. B. SaaS), wenn Eigenerstellung keinen strategischen

Mehrwert liefert oder Betriebskosten unverhältnismäßig hoch sind [1].

Retain:

Beibehaltung einer Komponente (vorerst keine Migration), z. B. aus Sicherheitsgründen, wegen Abhängigkeiten oder fehlender Priorität [1].

Retire:

Außerbetriebnahme einer Komponente, wenn kein geschäftlicher Nutzen (mehr) besteht oder Betrieb unverhältnismäßig teuer ist [1].

Resilienz:

Fähigkeit eines Systems, Störungen zu tolerieren und sich zu erholen; häufig eng verbunden mit Observability, Redundanz und klaren Fehlergrenzen.

Technische Schulden:

Metapher für langfristige Folgekosten kurzfristiger Kompromisse (z. B. fehlende Tests, starke Kopplung, veraltete Abhängigkeiten), die Wartbarkeit und Änderbarkeit beeinträchtigen.

Ausgewählte Quellen für dieses Exposé

Die folgenden Quellen wurden in dieser Arbeit zentral verwendet:

- Amazon Web Services. *Was ist eine Cloud-Migrationsstrategie?* 2026. URL: <https://aws.amazon.com/de/what-is/cloud-migration-strategy/> (besucht am 13.01.2026)
- Mark Richards und Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. URL: <https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/> (besucht am 13.01.2026)
- Neal Ford u. a. *Software Architecture: The Hard Parts*. O'Reilly Media, 2021. URL: <https://www.oreilly.com/library/view/software-architecture-the/9781492086888/> (besucht am 13.01.2026)
- Cornelia Davis. *Cloud Native Patterns: Designing Change-tolerant Software*. Manning Publications, 2019. URL: <https://www.manning.com/books/cloud-native-patterns> (besucht am 13.01.2026)
- MDN Web Docs. *Progressive web apps (PWAs)*. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (besucht am 13.01.2026)
- Spring. *Spring Boot Reference Documentation*. 2024. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (besucht am 13.01.2026)
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2023. URL: <https://www.postgresql.org/docs/current/> (besucht am 13.01.2026)

- National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Techn. Ber. NIST, 2023. URL: <https://csrc.nist.gov/pubs/ai/100-1/final> (besucht am 13.01.2026)

Quellen und weiterführende Literatur

- [1] Amazon Web Services. *Was ist eine Cloud-Migrationsstrategie?* 2026. URL: <https://aws.amazon.com/de/what-is/cloud-migration-strategy/> (besucht am 13.01.2026).
- [2] Mark Richards und Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. URL: <https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/> (besucht am 13.01.2026).
- [3] Neal Ford u. a. *Software Architecture: The Hard Parts*. O'Reilly Media, 2021. URL: <https://www.oreilly.com/library/view/software-architecture-the/9781492086888/> (besucht am 13.01.2026).
- [4] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Techn. Ber. NIST, 2023. URL: <https://csrc.nist.gov/pubs/ai/100-1/final> (besucht am 13.01.2026).
- [5] MDN Web Docs. *Progressive web apps (PWAs)*. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (besucht am 13.01.2026).
- [6] Spring. *Spring Boot Reference Documentation*. 2024. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (besucht am 13.01.2026).
- [7] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2023. URL: <https://www.postgresql.org/docs/current/> (besucht am 13.01.2026).
- [8] Cornelia Davis. *Cloud Native Patterns: Designing Change-tolerant Software*. Manning Publications, 2019. URL: <https://www.manning.com/books/cloud-native-patterns> (besucht am 13.01.2026).

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Artikels wurden KI-Tools (GitHub Copilot, ChatGPT) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt bei der Autorin.